

PROJECT AEGIS

Sécuriser un pipeline CI/CD de bout en bout, du premier commit au déploiement.

Niveau visé : Junior / Débutant (0-1 an) | Durée estimée : 3 à 4 semaines, à temps partiel | Format : projet solo, autonomie totale

Contexte

Une jeune entreprise SaaS déploie son application manuellement, sans aucun filet de sécurité : pas de scan, pas de contrôle, pas de traçabilité. Un secret a déjà fuité une fois dans un commit public. Le fondateur te confie une mission claire : construire depuis zéro un pipeline CI/CD qui empêche automatiquement tout code dangereux d'atteindre la production, sans ralentir les livraisons. Tu es libre de repartir d'une application existante (par exemple un de tes projets fullstack) : l'objectif n'est pas de coder une nouvelle feature, mais de blinder ce qui existe déjà.

Objectifs pédagogiques

- Comprendre et appliquer le principe du **Shift Left Security** : détecter le risque le plus tôt possible, pas après coup.
- Savoir construire un pipeline CI/CD multi-étapes de A à Z (build → test → scan → déploiement).
- Être capable de lire un rapport de vulnérabilités et de trier vrais positifs et faux positifs, sans paniquer ni tout ignorer.
- Sécuriser une image de conteneur Docker avant sa mise en production.

Compétences visées

Compétence	Ce que tu dois savoir faire / outils
CI/CD	GitHub Actions ou GitLab CI — construire un pipeline avec au minimum les étapes build, test, scan, déploiement.
SAST	SonarQube ou Semgrep — analyse statique du code source pour détecter les patterns dangereux.
SCA	Snyk ou Dependabot — scanner les dépendances tierces pour détecter les CVE connues.
Secret detection	gitleaks — empêcher qu'une clé API ou un mot de passe parte dans un commit.
Container security	Docker + Trivy — scanner une image avant de l'envoyer en production.

Livrables attendus

- Un dépôt Git public contenant une application fonctionnelle (reprise ou nouvelle).

- Un pipeline CI/CD opérationnel avec au moins 4 étapes de sécurité automatisées.
- Un rapport de sécurité (PDF ou Markdown) : vulnérabilités trouvées, corrigées, et celles jugées acceptables — avec justification.
- Un README expliquant l'architecture du pipeline et les choix effectués.

Contraintes techniques obligatoires

- Le pipeline doit **bloquer automatiquement** tout déploiement en cas de vulnérabilité critique détectée.
- Aucun secret ne doit apparaître en clair dans l'historique Git (preuve : scan gitleaks propre).
- L'image Docker finale ne doit présenter aucune vulnérabilité « critical » non traitée.

Critères d'évaluation

Critère d'évaluation	Poids
Pipeline fonctionnel et entièrement automatisé	40%
Pertinence du tri des vulnérabilités (vrai/faux positif)	25%
Qualité et clarté du rapport de sécurité	20%
Qualité de la documentation (README)	15%

Ce sujet est conçu pour être réalisé en autonomie complète. Aucune correction type n'existe : documente tes choix, assume-les, et sois capable de les défendre à l'oral face à un recruteur comme tu le ferais face à un jury.